EPSR-test-ex24 epsrwts: interactive neutron weights

Program(s): epsrwts | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

Exercises `epsrwts`, the older interactive counterpart of `nwts`: every isotopic composition, exchange flag, substitution, mixing fraction, file name and normalisation choice is typed at prompts. It writes the same kinds of neutron weights files (single sample, first- and second-order differences) used by `epsr` [1,2], and the case drives all its input-validation loops from scripted standard input.

Code exercised

Fortran (original, fortran_src/):

- `epsrwts.f`: `epsrwts` (prompts with `iostat` re-asking, isotope abundances, exchange averaging, `<base>tot*.wts`, `dif`, `xx/xy/yy`; isotope-resolved intramolecular H-H weights)
- `ato_manipulation_rev4.f`: `scatt_len` (table `sears91_gudrun.inc`)
- `ato_read_and_save.f`: `minitc`, `moloutc`; `file_and_directory_commands.f`: `get_full_filename`

Rust (port, rust/epsr-core/src/):

- `tools/epsrwts.rs`: `run`, `rd_int_err`, `rd_real_err` (re-asking on invalid answers)
- `tools/nwts.rs`: `scatt_len` (shared); `ato.rs`: `minitc`, `moloutc`

Physical system

The EPSR-test mixture of 200 DMSO and 400 water molecules (3200 atoms; types S, C, O, H, O1, H1). The scripted experiments: `case1` - one sample, per atom, D on methyl H and on exchangeable H1, not normalised; `case2` - two samples per molecule with two-isotope C and H compositions, H and H1 substituted, normalised totals and difference; `case3` - three samples (second-order difference, mixing fraction 0.5) per molecule; `case4` - three samples per atom with non-exchanging H and H1 carrying the same 1H/2H mixture, S and H substituted, mixing fraction 0.3. Scattering lengths after Sears [3]; Faber-Ziman pair weights [4].

Inputs

`200DMSO400Water.ato` from EPSR-test and four answer files `case{1..4}.stdin`. `case2` deliberately gives an invalid sample count (5), an invalid exchange flag (7), an unknown mass number (99), a non-numeric answer (`x`) and a base name with leading blanks and extra words (`HD2 extra`); it also names the `.ato` file without extension.

How the accuracy was verified

`commands.txt` (4 program calls, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in 1 pass.

One pass of four `epsrwts` calls; 17 files compared: `D1tot.wts`, `HD2tot1/2.wts`, `HD2dif.wts`, `X3tot1/2/3.wts`, `X3xx/xy/yy.wts`, the six `X4*.wts` files and the rewritten `.ato` file. All re-asked answers lead to the same output names and values in both codes.

Notes

Reproduced Fortran behaviour: for a component with two isotopes only the last abundance is normalised (README).

References

[1] A. K. Soper, "Partial structure factors from disordered materials diffraction data: An approach using empirical potential structure refinement", Phys. Rev. B 72, 104204 (2005).

[2] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).

[3] V. F. Sears, "Neutron scattering lengths and cross sections", Neutron News 3(3), 26-37 (1992).

[4] T. E. Faber and J. M. Ziman, "A theory of the electrical properties of liquid metals III. The resistivity of binary alloys", Phil. Mag. 11, 153-173 (1965).

[5] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.